Mise en route Intellij. Il faut que java puisse accesder aux fichiers hamcrest-core2.2-sources.jar junit-4.13.2-sources.jar, dans les librairies. Ces fichiers se trouvent à la racine. Normalement ca devrait fonctionner direct, mais il se peut que vous deviez maniper : un clic droit et selectionner "add as library". Normalement le dossier tests devrait apparaitre en vert, c'est la couleur des tests, sinon Faire un clic droit sur tests et selectionner "add as test source root" voire aussi un clic droit sur src et selectionner "add as source root",

Autre méthode possible en cas de problémes : Ouvrez par exemple vect.test, et cliquez sur le "junit" qui apparait en rouge dans la ligne au tout début "import org.junit.*;", une ampoule rouge va s'allumer. Cliquez sur cette ampoule, l'idea vous suggére alors (en bleu) de cliquer sur add Junit 4 to class Path, vous cliquez la dessus, puis OK sur le bandeau suivant qui apparait, et ca résoud le problème.

Gérer la correction et le squelette. Votre archive contient à la foit le squelette complété et le squelette à compléter c'est pratique. Si Intelij se met à crier que les classes sont définies deux fois. Il faut sélectionner celui des dossier src et test dont vous ne voulez pas, faire un clique droit, "mark directory as", "excluded".

6.1 VectTest.java

JUnit est un outil permettant d'écrire et d'exécuter des tests unitaires sur des programmes Java.

Il est disponible à l'adresse `http://www.junit.org/`. Il peut aussi s'utiliser avec Intelij. Un test en JUnit 4 est une méthode annotée par `@Test`. Les méthodes de test sont généralement regroupées en une classe dédiée aux tests. Ouvrez le fichier VectTest.java du repertoire tests. Si votre installation est correcte vous devez pouvoir éxecuter ce fichier, ce qui provoquera l'execution de tout les tests contenu dans la classe VectTest,

Question 5 Sur quoi porte les teste de VectTest ?

Question 6 Pourquoi sont il pour la pluspart marqué en rouge avec une mention "échec".

Question 7 Est-ce que les méthode de Vect, d'instantiation, add,sub, et egale de Vect sont compliquées ? que font elles ?

Question 8 Pourquoi vous a t'on mis cette quantité impressionantes de tests ?

Question 9 A quoi servent tout les attributs initialisés par la classe VectTest ?

Question 10 Comment faire passer les tests du rouge au vert ?

Question 11 Que pensez vous de l'utilité des tests sur normalize, et pourquoi sont ils tous devenus vert sans corrections supplémentaires ?

6.2 BolideTest.java

Question 12 : Que testent les méthodes void bolideZero(), void bolidePos() , void bolideNeg() ? Verdir !

Qu'est ce que les "Preambule" Le corps d'une méthode de test comporte en général trois parties :

- le *préambule*, qui permet de créer les objets et de les amener dans l'état nécessaire pour le test ;
- le *corps de test*, dans lequel la méthode à tester est appelée sur les objets créés ;
- l'*identification*, qui permet de délivrer le verdict du test (succès ou échec) en vérifiant un ensemble de propriétés (assertions) sur l'état des objets après le test.

Il est possible de grouper les tests ayant un préambule commun (c'est-à-dire devant être exécutés dans le même état) en une classe et de définir une méthode qui exécutera ce préambule avant chacun des tests de la classe. Cette méthode doit être annotée par `@Before`.

Question 12 bis A quoi sert le préambule de BolideTest, on aurait pu se passer de préambule ? pourquoi le préambule ?

Question 13 Que teste les 9 methodes suivantes dont le nom commence par calculeAcceleration ? En déduire une spécification, verdir.

Question 14 Que teste les 2 methodes suivantes dont le nom commence par AccelereDe ? En déduire une spécification, verdir.

Question 15 Que teste les 2 methodes suivantes dont le nom commence par stop ? En déduire une spécification, verdir.

Question 16 Idem pour les derniéres méthodes

6.3 CircuitTest

Question 17 Que déduisez vous des test Circuit simple ?

Question 18 Que déduisez vous des test "HorsLimite"

Question 19 Que déduisez vous des test "CarteTropGrande", "CarteTropPetite" ?

Question 19 bis Programmer le constructeur de Circuit.

Question 20 Que déduisez vous des test DeplaceBolide

Question 20 Programmer deplacementBolide.

Question 21 Finalement, que fait gereClick ? Programmez.

Question 22 Pour les "mordus" : colorier en vert le trajet juste effectué, comme ca se passe dans la démo que vous avez fait au début. C'est pas très compliqué, si vous le faites vous allez vous sentir super pro.

7 TD3 :Projet

Il y a un TD entier consacré au projet. Il consiste principalement a mettre en place le diagramme des classes du projet. C'est un travail réalisé avec tout le groupe, chaque groupe aura potentiellement un diagramme assez différent, suivant la compréhension plus au moins préciser qu'ils on du sujet, et que leur prof également a du sujet. On pourra commencer par répondre aux questions des étudiants sur la compréhension du projet, s'il y en a. Je met un certain nombre d'infos dans un howto générique. Je parle de ces info en cours donc y a pas trop besoin de passer dessus durant le TD projet.

7.1 Howto générique

Je met un certain nombre d'infos ici, pour que mes chargés de TP les lisent aussi, en plus du cours. Je parle de ces info en cours donc y a pas trop besoin de passer dessus durant le TD projet.

Calcul de la note Pour la session1, le projet compte pour 25% , partiel 25% et examen 50%. Pour la session2, il compte pour 33% et examen rattrapage 66%. Notez bien que si vous ne travaillez pas sur le projet, votre note de projet reste, et risque de vous plombez pour le passage. Comme le partiel ne compte plus, ce qu'on doit réviser pour le rattrapage inclus le programme du partiel.

7.1.1 Déroulement

Le projet démarre avec une semaine qui lui est dédiée : cours + TD, deux semaines aprés les partiels. Je prie le groupe dont le TD est décalé de une semaine, de ne pas se plaindre d'avoir moins de temps, en effet, il peut tout à fait commencer aprés le cours, avant le TD dedié au projet, car le tout début du projet porte sur la prise en main de swing, avec simplement l'affichage du plateau de jeu, tandisque le TD dedié, lui, a pour but de produire avec les étudiants présents, une ébauche du diagramme de classe global.

Vous avez a peu près 5 semaines jusqu'à la soutenance. Le cours est entiérement structuré pour maximiser ce laps de temps. Il n'est malgré tout pas énorme, et vous aurez d'autres projets avec d'autre matières, il est donc essentiel de démarrer immédiatement. Le projet est à réaliser en binome de deux étudiants d'un même groupe à peu près du même niveau [1]. Il est déconseillé de le faire tout seul, c'est moins drôle, et vous n'apprenez pas à travailler en groupe. Il n'est pas autorisé de faire le projet à trois, car sur trois il y en a souvent un qui ne fais rien. Vous saisirez vos binomes dans ce fichier partagé :

`https://docs.google.com/document/d/1a0a7xA-j2NIUAr1qwjue_VL1KtK9hsH01K8oEsOZqqQ/edit`

Celui-ci servira également pour vous informer des heures de passage, pendant les soutenances.

Votre projet doit être hébergé sur un dépôt git. Vous n'êtes pas autorisé à le faire sur github. On vous demande de le prendre chez nous sur le gitlab de l'université `https://gitlab.lisn.upsaclay.fr/users/sign_in` surlequel vous avez un compte.

7.1.2 Intruction pour le dépot git

Création du projet à faire par l'un des membres du binôme.

1. Allez sur l'interface web du gitlab de l'université : https ://gitlab.dsi.universite-paris-saclay.fr/ et connectez-vous (avec vos identifiants habituels de l'université).
2. Cliquez en haut à droite : new project, puis sélectionnez create blank project.
3. Donnez un nom à votre projet.
4. Assurez-vous que le niveau de visibilité private est sélectionné, puis confirmez la création.

Ajouter son binomes et son prof de TP, à réaliser dans l'interface web du projet.

1. Dans le menu, sélectionnez "project information", puis "members".
2. Cliquez sur invite member et intégrez le deuxième membre du binôme, avec le rôle maintainer.
3. Recommencez et intégrez votre encadrant(e) de TP, avec un rôle reporter (ou supérieur), puis de même avec frederic.gruau, je pourrai venir visiter en cas de problème.

Note : vous ne pouvez inviter que quelqu'un qui s'est déjà connecté à ce gitlab.

Connecter Git et IntelliJ, Import du projet, à faire par chaque membre du groupe et sur chaque ordinateur utilisé.

1. lorsque vous faites "New-¿Project From Version Control" dans IntelliJ
2. Une fenêtre s'ouvre, dans la barre latérale à gauche cliquer sur GitLab. Dans la barre prévue à cet effet entrer l'URL suivante : `https://gitlab.dsi.universite-paris-saclay.fr/`
3. Cliquer sur Generate Token, se connecter à GitLab et accepter ensuite les autorisations souhaitées sur GitLab.
4. Copier la clé générée et l'insérer dans IntelliJ dans le champ de texte prévu à cet effet.
5. Cliquer sur clone et normalement ça fonctionne !
6. Il est possible qu'IntelliJ vous redemande le token lors de votre premier push

Autre possibilité de problémes :

1. Dans Version control : Git, il est possible que le message rouge Git is not installed apparaisse. Dans ce cas, systématiquement à droite sur la même ligne il y a un lien cliquable Download and install. Cliquer dessus et l'installation se fait automatiquement. Après quelques secondes de patience, reprendre les étapes du TP.
2. lors de la création de la clé Token, sélectionner role as Developer / Maintener et non pas Guest comme par défaut.

Initialisation des fichiers, à faire par l'un des membres du groupe.

1. Dans le projet IntelliJ, faites un clic droit new, directory, et nommez src le dossier créé.
2. Faites un clic droit sur src, puis sélectionnez mark directory as, et sources root.

1. La formation des binomes est supervisée par votre TPman, celui ci peut autoriser un regroupement intergroupe, pour éviter de faire le projet seul, ou parceque vous avez un bon ami dans un autre groupe

3. Ajoutez dans le dossier src vos fichiers .java. Si l'interface vous propose des les ajouter à Git, acceptez.
4. Poussez les nouveaux fichiers sur le dépôt Git : dans le menu git, allez chercher les actions commit et push (l'action commit ouvrira dans l'interface une fenêtre où vous devez écrire un message décrivant le contenu de la modification que vous enregistrez).

Travailler avec git À réaliser par chaque personne travaillant sur le projet.

1. Au début de chaque session de travail, récupérez les fichiers du dépôt partagé pour mettre à jour vos fichiers locaux avec pull. Pour cela : cliquez dans le menu git, puis pull. Recommandation : c'est généralement une bonne idée, dans modify options, d'allez sélectionner –rebase.
2. Après chaque modification significative, enregistrez (localement) votre travail avec commit. Dans le menu git, sélectionner commit, et entrer un message décrivant la modification.
3. À la fin de chaque séance (voire plus souvent), publiez vos commits sur le dépôt partagé avec push. Dans le menu git, sélectionner push, et valider.

Pour éviter les interférences entre les modifications faites par plusieurs personnes travaillant en parallèle sur le même projet, il vaut mieux souvent publier ses modifications (commit/push) et toujours commencer par récupérer les modifications des partenaires (pull) avant de commencer à coder.

Intellij propose une interface "code with me" qui permet de programmer simultanément à plusieurs en temps réel. Elle a été utilisé avec succés par certains de mes meilleurs étudiants, en 2022, qui m'en ont dit le plus grand bien.

7.1.3 Consignes

Faites-le dépot git dès le début, ce afin que nous puissions nous assurer que vous avancez. La mise à jour régulière de la version sur git fait partie des critères dans la note finale.

Il n'y a pas à proprement parler de "rendu" , votre rapport sera dans le README.txt de votre git, excepté le diagramme de classes que vous pouvez publier séparément, dans le git, sous forme d'un PDF. Il est préférable que vous dessinez un résumé du diagramme, qui explique vos choix de structures. Le diagramme généré automatiquement est trop complexe à lire et à comprendre. Vous pouvez aussi mettre une ou deux impressions ecran de votre rendu visuel.

Les soutenances ont lieu à votre horaire de TP, dans votre salle habituelle de TP, la semaine d'avant les examen De cette manière, la maitrise java que vous aurez acquis grâce au projet, vous servira aussi pour réussir l'examen : coup double. Nous assignerons un horaire précis dans le fichier des binomes. Les soutenances sont rapides, entre 10 et 15 minutes. Pendant qu'un groupe passe, le suivant s'installe, et les autres attendent dans une autre salle à côté. L'un des objectifs principal de la soutenance est de vérifier que vous êtes bien l'auteur de votre projet, vous serez noté sur les parties que vous maitrisez. Si vous ne pouvez pas expliquer le code, on considére que ce n'est pas votre code, vous avez 2 sur 20, et cela arrive. En effet l'expérience montre qu'il est en fait impossible de simuler être l'auteur d'un code qu'on a pas produit.

Le README.txt détaille :

1. Les parties du sujet que vous avez traitées.
2. Vos choix d'architectures.
   Les choix d'architecture font partie de ce que nous allons évaluer : donnez un peu de détail sur l'organisation de vos classes et le partage des rôles. Utiliser les différentes techniques vues en cours et en TP.
3. Les problèmes qui sont présents et que vous n'avez pas pu éliminer.
4. Les morceaux de code écrits à plusieurs ou empruntés ailleurs.
   Sans cette information, cela s'appelle du plagiat, et est considéré comme une fraude par le réglement de l'université.

Intellij dans sa version "ultimate" [2], vous permet de directement pondre tout le diagramme de classes, mais c'est dur à lire pour le correcteur. Si vous souhaitez avoir tout les points pour le diagramme de classe, il faut faire l'effort d'en faire une version résumée plus lisible. Nous allons tester votre application : le code source doit impérativement compiler sans erreur. *Si des morceaux de code ne sont pas finis, vous pouvez les mettre en commentaire. Une méthode non implémentée peut avoir un code vide, ou renvoyer une valeur arbitraire comme* `null`.

Présence à la soutenance Elle est obligatoire pour tous les membres du groupe car il nous faut valider l'implication réelle de chaque participant, et cela implique du présentiel ! Vous pourrez peut être difficilement profiter de la semaine de révisions pour faire le voyage chez vos parents, si ceux ci habitent trop loin.

7.1.4 L'évaluation

Les trois consignes (format et contenu du `README`, code source qui compile, presence) sont impératives ; leur non-respect équivaut à la note zéro. La note finale sera basée principalement sur l'avancement de votre projet, le premier but de votre présentation est donc de démontrer jusqu'ou vous êtes allé. Si rien ne s'éxecute vous avez 2 sur 20. Compte aussi : la qualité de la présentation, l'implication respective des deux membres, votre `README`, la maitrise du code dont vous prétendrez être l'auteur (comment vous répondez aux questions), la qualité du code, la présence de tests junit, la clarté de la présentation, l' habillage graphique, les commentaires et en particulier la description des paramètres de chaque méthode, le fait de répartir les classes dans trois packages : modele, vue et controlleur.

2. cette version est payante, mais avec votre email a paris-saclay vous y avez accés gratuitement

7.1.5 Cause principale d'échec

De commencer a s'y mettre trop tard. Vers la fin du semestre vous êtes assommé de trucs à faire, car il n'y a pas seulement le projet de POGL. Pour réussir sans douleur, et profitez de cette opportunité que vous avez de devenir des programmeurs "pros", commencez à travailler exactement maintenant en imaginant que la soutenance est la semaine prochaine. Chaque année, un bon quart des binomes plantent pour cause de mauvais timing. Je garanti que vous êtes tous capable de réussir au moins 12 sur 20 si vous démarrez à temps.

8 Le projet "le désert interdit"

Ce projet de programmation vous invite à construire une version informatique et un peu simplifiée du jeu *Le désert interdite* de Matt Leacock.

Votre équipe d'aventuriers a été envoyée en mission pour mettre à jour une cité ensevelie dans un ancien désert et retrouver une machine volante légendaire. La rumeur précise même qu'elle fonctionnerait à l'énergie solaire. Mais au moment d'arriver à destination, une tempête de sable inattendue entraine un atterrissage forcé de votre hélicoptère. À présent bloqués dans cette étendue immense et exposés à une insatiable tempête, votre seul espoir d'en réchapper est de mettre à jour la cité rapidement pour trouver les pièces de la machine volante et la reconstruire. Mais si un membre de l'équipe meurt de soif ou si la tempête devient trop forte, toute l'équipe échoue et fera de belles reliques dans le désert interdit...

Règles du jeu

La compréhension des règles n'est pas évidente, en cas de difficultés, vous trouverez un texte illustré et une vidéo sur les vraies règles sur les urls suivants :

```
https://cdn.1j1ju.com/medias/5f/ad/
 62-le-desert-interdit-regle.pdf
https://videoregles.net/videoregle/
 le-desert-interdit
```

Les joueurs doivent récupérer 4 éléments de la machine volante, pour s'envoler avec : le moteur, l'hélice, le gouvernail, le capteur d'énergie solaire. Le désert est formée par un ensemble de zones disposées en grille 5X5. On distingue les zones particulières suivants :

- L'oeuil de la tempête (1 zone)
- l'endroit du crash de l'hélicoptère.(1 zone)
- La piste de décollage (1 zone)
- Pour chaque pièces, deux zones indiquant respectivement sur quelle ligne et sur quelle colonne placer chacunes des quatres pièces.(8 zones)
- des oasis (3 zones)
- des tunnels (2 ou 3 zones)

Chaque zone peut être explorée ou pas, lorsque on l'explore, la zone montre sa face explorée, et ne pourra plus être explorée a nouveau. Sur chaque zone, exceptée l'oeuil, il peut y avoir un certain nombre de tonnes de sable dessus. Lorsque une zone a deux tonnes de sable ou plus, elles est bloquée, un joueur ne peut plus s'y déplacer. L'oeuil de la tempête est toujours bloqué. Un joueur peut ramasser et donc avoir en main des pièces de la machines, parmis les quatres. Chaque joueur porte une gourde d'eau, avec un niveau entre 0 et 5.

Simplification possible Au lieu de 8 zones d'indice d'objets, on peut décider d'avoir directement 4 zones d'objets, je n'ai pas fait cela car cela le rends beaucoup plus facile à jouer et donc moins intéressant. C'est une simplification que pourront adopter les binômes qui n'auront pas vraiment envie de jouer, et qui sont un peu effrayés par le projet.

Situation initiale : Parmi les zones spéciales, on peut voir seulement L'oeuil de la tempête (qui se trouve au centre), la zone du crash et les trois oasis ; Pour les autres zones spéciales, il faudra les explorer pour les mettre à jour. Il y a initialement une tonnes de sable sur 8 cases disposée en losange (voir plus loin les coordonnées précises).

Le tour d'un joueur . Le joueur à qui c'est le tour effectue au maximum quartes actions à choisir parmi :

1. Se déplacer vers une zone adjacente non bloquée (haut, bas, gauche, droite),
2. Creuser, enlever une tonnes de sable de la zone sur laquelle il se trouve ou une zone adjacente
3. Explorer la zone sur laquelle il se trouve
4. Ramasser une pièce de la machine volante

En plus, le joueur [3] peut décider de donner ou de prendre de l'eau à un autre joueur qui se trouve sur la même case, sans que cela ne compte pour une action. La tempête de sable à un certain niveau modélisé par un nombre réel initialement fixé a deux, et qui peut augmenter. A la fin de chaque tour de joueurs, des "action du désert" en nombre égal a la partie entière du niveau actuel de tempête, sont déclenchée parmi trois possibles,

- L'action "le vent souffle", paramétrisée par une direction parmi les quatre possibles, et une force qui est un nombre entier f entre 1 et 3.
- L'action "vague de chaleur", qui va obliger chaque joueur à boire dans sa gourde, et donc le niveau de leur gourde va descendre d'un cran.
- l'action "la tempête se déchaîne" qui va augmenter le niveau de la tempête en ajoutant 0.5.

L'action du vent d'une force f et d'une direction d déplace f tuiles d'une case vers l'oeuil de la tempête, et rajoute une tonne de sable sur chacune d'elle. (voir le pdf explicatif, ou la vidéo, ou le cours). Cela va aussi avoir comme effet de bord de déplacer l'oeuil. Il se peut que on ne puisse déplacer seulement 1 ou 2 tuiles, voire 0. Les probabilité respective vent, chaleur, déchaînement sont à déterminer pour ajuster la difficulté.

3. dans le vrai jeu cette action se fait a tout moment par chaque joueur

Fin de partie La partie est gagnée lorsque les 4 pièces sont ramassées, les joueurs se trouvent sur la piste de décollage, et celle ci est désensablée, ils peuvent alors s'envoler a bord de la machine fantastique. La partie est perdue dans trois cas :

1. Si il y a plus que 43 tonnes de sables sur le plateau l'équipe est définitivement ensablée
2. Si l'un des joueurs a besoin de boire mais n'a plus d'eau, il meurt de soif.
3. Si le niveau de la tempête atteint 7.

Organisation

L'application sera organisée selon une architecture Modèle-Vue-Contrôleur (MVC), et vous pouvez vous inspirer du fichier `Conway.java` disponible dans l'archive des TPs Les différentes sections du sujet vous proposent de mettre progressivement en place les différents éléments du jeu, et suggèrent un ordre dans lequel les traiter. Précisons toutefois qu'il est toujours utile de réfléchir en amont à la manière dont les éléments suivants pourront s'insérer dans votre code.

8.1 Promenade sur un chameau.

Dans un premier temps nous considérons uniquement le désert sans aucun joueur et sans zones spéciales, mis à part l'oeuil qui est nécessaire pour faire souffler le vent.

1. Créer ce modèle réduit dans lequel seules l'oeuil et les zones quelconques sont présentes.
   Nous vous encourageons à inclure des méthodes `toString()` permettant de représenter les différents objets, des affichages rendant compte des différents événements, et des tests unitaires quand cela est pertinent.
2. Créer une vue affichant le desert.
3. Placez une tonne de sable sur les cases de coordonnées (0,2),(1,1),(1,3),(2,0),(2,4),(3,1),(3,3), (4,2), vérifiez que l'on peut voir la quantité de sable sur chaque zone, et la quantité totale qui est 8.
4. Ajouter un bouton « fin de tour ». Un clic sur ce bouton doit déclencher les actions du désert, parmi souffler, ou déchaîner, vous devez donc voir votre désert s'ensabler progressivement, et l'oeuil bouger.
5. Tester la fin de partie par ensablement, puis par tempête surpuissante.

8.2 Y a du sable dans mon sandwich !

Nous allons maintenant inclure des joueurs, qui pourront se déplacer, puis creuser.

1. Étendre le modèle avec un joueur et les méthodes permettant de faire se déplacer le joueur de sa zone courante à une zone adjacente.
2. Faire en sorte que la vue représente le joueur sur sa zone.
3. Étendre la vue et le contrôleur avec des moyens d'ordonner le mouvement du joueur.
   Au choix : clics de boutons, frappes au clavier, clics sur la zone cible, autre.
4. Étendre le contrôleur pour que le joueur puisse réaliser au maximum quatre actions avant de cliquer sur le bouton « fin de tour ».
5. Étendre l'ensemble pour permettre plusieurs joueurs. Le contrôleur doit faire agir les joueurs à tour de rôle, et insérer les actions du désert à la fin du tour de chaque joueur.
6. Étendre l'ensemble pour ajouter aux coups du joueur l'action de creuser sur la zone sur laquelle il se trouve ou une zone adjacente, en choisissant une interface utilisateur cohérente avec celle permettant le déplacement.
7. Faire que la vue indique le joueur dont c'est le tour et le nombre d'actions auxquelles il a encore droit.

8.3 Un pastis s'il vous plaît.

On va rajouter le rôle de l'eau dans le jeu. Les joueurs démarrent avec une gourde pleine (niveau5). Lorsqu'un joueur explore une case oasis, tout les joueurs qui se trouve dessus ont leur niveau de gourde qui montent de 2. Rappel : on ne peut explorer qu'une seule fois. Nous vous proposons l'ordre d'implémentation suivants, dont la logique est de pouvoir tester dés que possible.

1. Étendre la vue d'un nouveau panneau représentant les joueurs, et y indiquant leur niveau de gourde
2. Rajouter l'action "vague de chaleur" dans les action du désert possible
3. Tester la fin de partie par mort de soif.
4. Lorsqu'un joueur explore une case oasis, tout les joueurs qui se trouvent dessus ont leur niveau de gourde qui montent de 2.
5. L'une des trois cases oasis est un mirage (y a pas d'eau), et cela se voit que après son exploration.
6. Les joueurs qui se trouvent dans un tunnel n'ont pas besoin de boire lors des vagues de chaleurs.
7. Pour finir, implémenter le fait que les tunnels communiquent, le vrai jeu à trois zones tunnel, cependant si vous ne vous déplacer pas en cliquant, vous en prendrez que deux pour simplifier la sélection du déplacement en tunnel.

8.4 Envol de la machine fantastique

Pour finir de rendre le jeu opérationnel, nous allons ajouter la zone de crash, d'où on part, la piste de décollage ou il faut se rendre à la fin, et les cases d'indice pour trouver les pièces de la machine.

1. Étendre le modèle pour inclure les nouvelles zones spéciales
2. Étendre le contrôleur pour inclure l'action de ramasser un objet.
3. Étendre la vue pour représenter les objets ramassés.
4. Tester la fin de partie gagnante.

8.5 Le vrai jeu

Voici les deux aspects manquant pour modéliser le jeu complet, ils sont optionnels, seulement pour les binômes mordus. Notez que selon les choix de conception que vous avez faits jusqu'ici, certaines de ces propositions pourront demander des remaniements.

Equipement Lorsqu'on explore une zone, on récupére aussi une carte d'équipement parmis les suivantes :

1. Le jetpack permet de se déplacer sur n'importe quelle case, et de plus en emenant un autre joueur.
2. Le bouclier solaire pour se défendre contre les vagues de chaleur,
3. Des blaster qui permettent d'enlever en une seule action tout le sables se trouvant sur une zone.
4. Un appareil pour directement voir ce que il y a dans une zone sans avoir besoin de faire l'action explorer.

Vous pouvez gardez la carte jusqu'à ce que vous décidiez de l'utiliser. Une carte Équipement ne peut être utilisée que une seule fois, et que par le joueur qui la possède. Cependant, ces cartes peuvent être passées à tout moment à un joueur sur la même tuile. Il n'y a pas de limites au nombre de cartes équipements possédées.

Tas de cartes Aussi bien pour les équipements, que pour les actions du déserts, on utilise un tas de cartes du quel on retire progressivement des cartes pour savoir l'équipement obtenu, ou la prochaine action du désert. Pour éviter cette complication, jusqu'à maintenant, on tire les actions avec une certaine probabilité, mais en faisant cela, on autorise par exemple que la tempête se déchaîne un grand nombre de fois rapidement. Donc pour une modélisation plus précise il est nécessaire d'implémenter les tas de cartes.

Roles spéciaux Selon l'auteur du jeu, c'est la que cela devient palpitant.

1. L'archéologue peut enlever 2 tonnes de sable sur la même tuile pour 1 action.
2. L'alpiniste peut aller sur les tuiles bloquées (les tuiles ayant au moins 2 tonnes de sable). Elle peut aussi emmener un autre joueur avec elle à chaque fois qu'elle se déplace. Tous les pions sur la tuile de l'alpiniste ne sont jamais enlisés et peuvent quitter la tuile de l'alpiniste même s'il y a 2 marqueurs Sable ou plus.
3. L'explorateur peut se déplacer, enlever du sable et utiliser les ≪ Blasters ≫ diagonalement.
4. La météorologue peut dépenser des actions pour tirer, à la fin de son tour, moins de cartes tempête (1 carte par action) que ne le nécessite le niveau actuel de la tempête de sable. Elle peut aussi dépenser 1 action pour regarder autant de cartes Tempête que son niveau actuel, puis en placer éventuellement une sous la pile. Les autres cartes sont remises sur le dessus de la pile dans l'ordre de son choix.
5. La navigatrice peut déplacer un autre joueur jusqu'à 3 tuiles non bloquées par action, tunnels inclus. Elle peut déplacer l'explorateur diagonalement et peut déplacer l'alpiniste sur les tuiles bloquées. Déplacée ainsi, l'alpiniste peut aussi utiliser son pouvoir et emmener un autre joueur (dont la navigatrice) !
6. La porteuse d'eau peut prendre 2 portions d'eau des tuiles ≪ Point d'eau ≫ déjà explorés pour 1 action. Elle peut aussi donner de l'eau aux joueurs sur les tuiles adjacentes gratuitement et à tout moment. Sa gourde commence avec 5 portions d'eau (au lieu de 4).

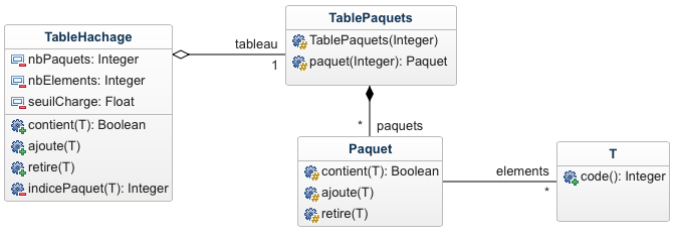
:

9 TD3 Hashtable : pre/post condition, invariant et Sequence Diagram

Dans ce TD, nous nous intéressons à la conception d'une structure de table de hachage, et en particulier à la manière dont les fonctionnalités principales se décomposent en enchaînements d'opérations des différentes classes composant la structure. Le thème abordés et la spécification des opérations. Il s'agira de préciser les valeurs que peuvent prendre les paramétres d'une méthode (préconditions) et les valeurs que prennent les valeurs retournées (postconditions). Ce TD illustre aussi comment la notion diagrammes de séquence introduite dans les diagrammes de cas peut aussi être utilisée pour la conception.

9.1 Tables de hachage

Voici un diagramme de classes représentant les différentes entités entrant dans la conception de notre table de hachage. Les classes TablePaquet et Paquet sont des classes internes privées utilisable seulement depuis "l'intérieur" de la table de hashage.



Dans la classe `TablePaquets`, les `paquets` sont stockés dans un tableau auquel on peut accéder par des indices. Dans la classe `Paquet`, les `elements` sont stockés dans une liste chaînée. Les invariants associés à ces classes sont :

— l'attribut `nbPaquets` est égal au nombre de paquets associés au `tableau`,
— l'attribut `nbElements` est égal au total des nombres d'éléments présents dans chaque paquet,
— le `tableau` ne contient aucun paquet égal à `null`, et aucun paquet ne contient l'élément `null`,
— tout élément `elt` de la table est dans le paquet d'indice `elt.code() % nbPaquets`. Ici, la fonction "code" renvoie un entier, et doit être définie.